

Circuitos combinatorios – Sumadores y restadores

Sumador completo (*Full Adder*)

El sumador completo, o *full adder*, en inglés (FA), es un circuito combinatorial que puede sumar tres números de un *bit* cada uno; es decir, que puede sumar tres *bits*.

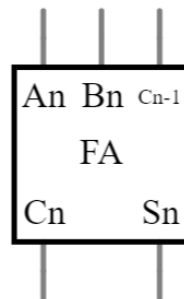
Generalizando los resultados obtenidos, se tiene:

- *Expresiones lógicas*

$$C_n = C_{n-1}(A_n \oplus B_n) + A_n \cdot B_n$$

$$S_n = A_n \oplus B_n \oplus C_{n-1}$$

- *Símbolo lógico*



2

Nótese la diferencia entre C_n y C_{n-1} :

C_{n-1} : *carry de entrada, correspondiente al carry anterior*

C_n : *carry de salida*

¿Cómo se podría hacer si se quieren sumar dos números que contengan más de un *bit* cada uno?

² Circuitos realizados en <https://www.falstad.com/circuit/>

Sumador en paralelo con *carry* en serie

Para realizar una suma de dos números de n bits, se pueden utilizar n sumadores que se ubicarán en paralelo, cuyo *carry* (o acarreo) de entrada obtendrá el *carry* del sumador anterior, es decir que los sumadores que se encuentran en paralelo pasan su *carry* de un sumador al siguiente en serie.

Ejemplo 1: sumador en paralelo con *carry* en serie

Se quiere realizar una implementación que realice la siguiente operación: $5_d + 1_d$ en binario.

Primero, hacemos la suma aritmética tal como se había visto:

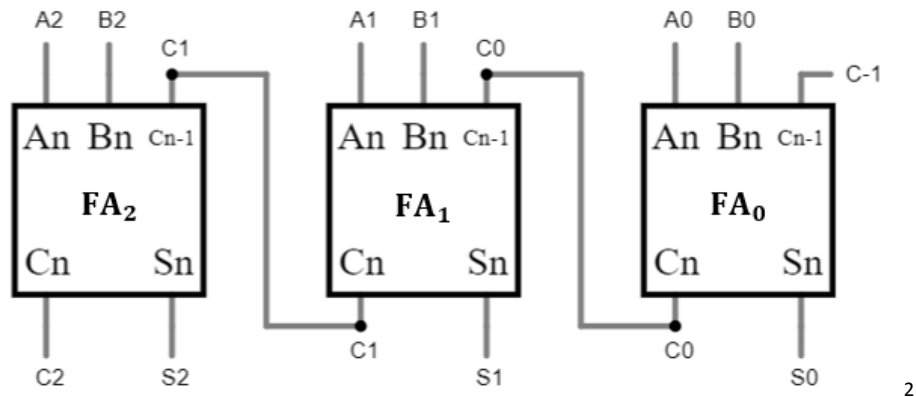
$$\begin{array}{r} 5_d = 101_b \\ 1_d = 001_b \\ \hline 110 \end{array}$$

Se propone que cada columna corresponda a un sumador completo, donde se suman dos números A y B .

$$\begin{array}{r} \begin{array}{ccc} A_2 & A_1 & A_0 \\ 5_d = & 1 & 0 & 1 \end{array} \\ \begin{array}{ccc} B_2 & B_1 & B_0 \\ 1_d = & 0 & 0 & 1 \end{array} \end{array} \quad + \quad \begin{array}{ccc} \begin{array}{c} 0 \\ 1 \\ 0 \end{array} & \begin{array}{c} 1 \\ 0 \\ 0 \end{array} & \begin{array}{c} 0 \\ 1 \\ 1 \end{array} \\ \hline 01 & 01 & 10 \end{array}$$

Los *carry* pasan a la siguiente columna y, por lo tanto, al siguiente sumador.

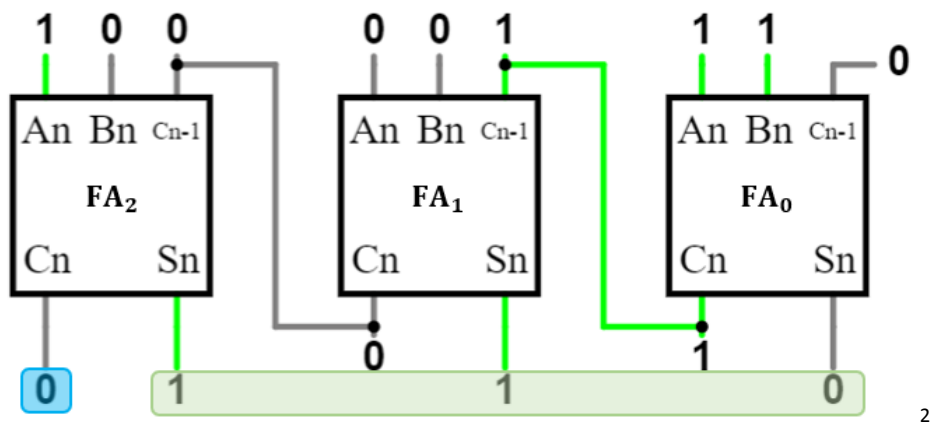
Entonces, como son tres columnas, se necesitarán tres sumadores. Se conectan los *carry* en serie, es decir que el *carry* de salida de un sumador pasa al próximo izquierdo.



Reemplazando los valores de los *bits* que se quieren sumar, se obtiene el siguiente resultado:

$C_{-1} = 0$, dado que se entiende que no hay un *carry* anterior al realizar la suma entre dos números positivos.

$$1 + 0 + 0 = 01 \quad 0 + 0 + 1 = 01 \quad 1 + 1 + 0 = 10$$



Los valores obtenidos en las salidas de los S_n corresponderán al resultado de la suma. En este caso, $S = S_2 S_1 S_0 = 110$, que es igual a 6 en decimal, y se corresponde con el valor esperado.

El último *carry* obtenido, $C_2 = 0$, indica que no hay un desbordamiento (*overflow*) en la cuenta.

Ejemplo 2: sumador en paralelo con carry en serie

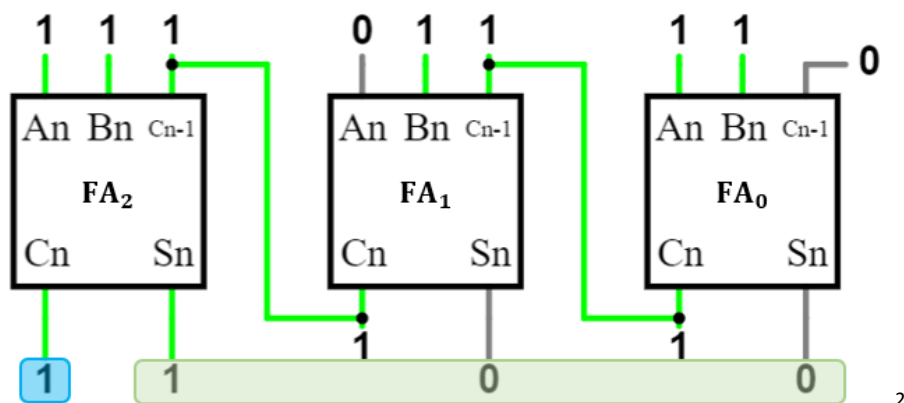
Se quiere realizar una implementación que realice la siguiente operación: $5_d + 7_d$ en binario.

Se deberá hacer el mismo planteo que antes:

$$5_d = \begin{matrix} A_2 & A_1 & A_0 \\ 1 & 0 & 1 \end{matrix}$$

$$7_d = \begin{matrix} B_2 & B_1 & B_0 \\ 1 & 1 & 1 \end{matrix}$$

Reemplazando los valores de las entradas, se obtiene lo siguiente:



En este caso, el último *carry* de la cuenta es 1 ($C_2 = 1$), lo que quiere decir que hay un desbordamiento (*overflow*) en la cuenta. Esto indica que se podría necesitar de al menos un sumador completo más para realizar la cuenta sin desbordamiento. Sin embargo, dado que se puede considerar que se está haciendo una cuenta sin signo, se podría tomar al último *carry* como parte del resultado de la cuenta. Por lo que, en este caso, el resultado de la suma sería $S = C_2 S_2 S_1 S_0 = 1100$, que si se pasa el resultado de la suma a decimal se tiene 12, que es el resultado esperado pues $5 + 7 = 12$.

¿Se podría realizar una resta entre dos números que contengan más de un *bit* cada uno?

Sí. Veamos las posibles implementaciones.

Restador a partir de sumador en paralelo con *carry* en serie

Para esta parte es importante recordar que cuando se quiere restar dos números $A - B$ es posible hacer esa cuenta sumando números con signo $(+A) + (-B)$. Esto nos da la idea de que se podría realizar la implementación solicitada a partir de sumadores completos.

Se vieron varias maneras de representar un número con signo en binario y se llegó a la conclusión de que la más conveniente es el complemento a 2. Por lo que se deberá ver cómo se puede implementar a través de sumadores completos una suma en complemento a 2.

Representación de números en binario con signo

En la siguiente tabla se puede ver la equivalencia en decimal del número escrito en binario con tres *bits* según la representación en la que se encuentra: signo y magnitud (SyM), complemento a 1 (Ca1) y complemento a 2 (Ca2):

	Con signo		
Binario	SyM	Ca1	Ca2
000	+0	+0	+0
001	+1	+1	+1
010	+2	+2	+2
011	+3	+3	+3
100	-0	-3	-4
101	-1	-2	-3
110	-2	-1	-2
111	-3	-0	-1
Rango 3 bits	$[-3; 3]$	$[-3; 3]$	$[-4; 3]$
Rango genérico		$[-(2^{n-1} - 1); (2^{n-1} - 1)]$	$[-(2^{n-1}); (2^{n-1} - 1)]$

De la tabla se deduce que cuando se quiere utilizar números con signo es más conveniente usar la representación de complemento a 2, dado que es el único caso donde el cero tiene un solo equivalente (el elemento nulo es único), por lo que el rango de representación es mayor.

Ejemplo 1: restador a partir de sumador en paralelo con carry en serie

Se quiere realizar una implementación que realice la siguiente operación: $5_d - 2_d$ en binario.

Primero, será conveniente plantear la cuenta tal como se había visto. Se tomará como A al número positivo, en este caso, $+5$, y como B al número negativo, en este caso, -2 .

Se escriben ambos números en binario con la misma cantidad de *bits*:

$$5_d = 101_b \text{ y } 2_d = 010_b$$

Ahora, se pasan ambos números a complemento a 2.

En el caso del $+5$, solo se agrega el *bit* de signo para indicar que es positivo:

$$5_d = 101_b \rightarrow +5_d = 0101$$

En el caso del -2 , se agrega el *bit* de signo, se invierten todos los *bits* (si son unos se cambian por ceros y si son ceros por unos) y, finalmente, se suma 1:

$$2_d = 010_b \rightarrow 0110 \rightarrow 1110 \rightarrow -2_d = 1110$$

Luego, se realiza la suma:

$$\begin{array}{r|l} & 1 \\ + & 0 \quad 1 \quad 0 \quad 1 \\ & 1 \quad 1 \quad 1 \quad 0 \\ \hline 1 & 0 \quad 0 \quad 1 \quad 1 \end{array}$$

Finalmente, se verifica el resultado y se obtiene que es equivalente a $+3$.

Después de haber planteado la cuenta como una suma con signo en complemento a 2, se está en condiciones de ver cómo plantear la cuenta para realizarla a través de sumadores.

¿Cuántos sumadores se necesitarán?

Dado que la cuenta es una suma que contiene tres columnas más la del *bit*, se usarán 4 sumadores completos.

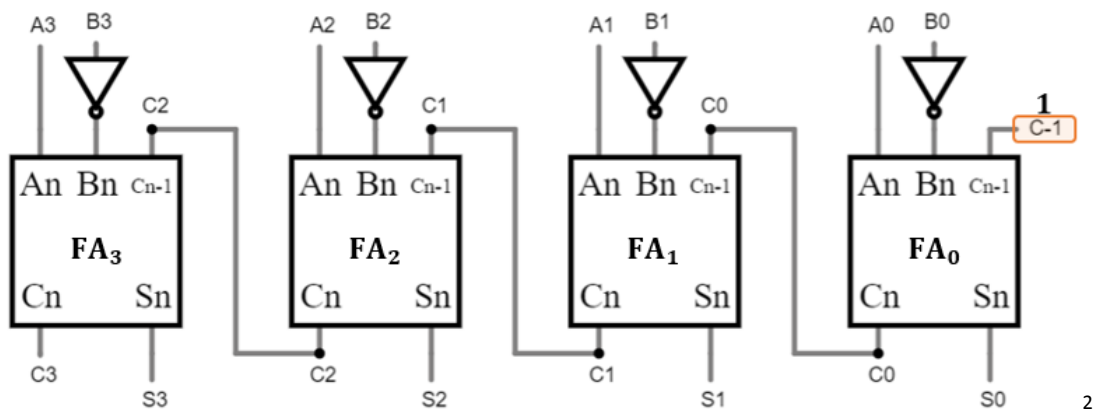
El número A , que es el positivo puede ir conectado directamente como en los casos anteriores de suma. Sin embargo, para el número B que es negativo se deberá pensar en utilizar alguna lógica adicional.

Teniendo el número B en positivo, lo primero que se hizo fue invertir todos los *bits*.

¿Cómo se podría hacer esto?

Si se piensa en el comportamiento de una compuerta *NOT*, cuando a esta le ingresa un '0' lo invierte por un '1', y viceversa. Por lo que se podría agregar a las entradas del sumador una compuerta *NOT*.

Además, para pasar el número a complemento a 2, aún falta sumarle 1 al último *bit* (B_0). Se podría hacer esto utilizando el *carry* $C_{-1} = 1$. Por lo que dicha implementación de forma genérica será:



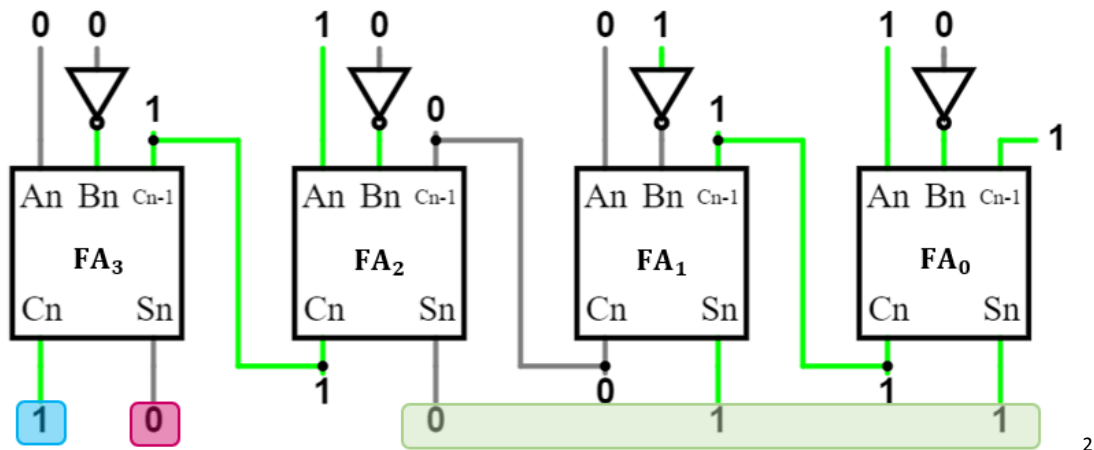
Entonces, A_n , B_n y C_{-1} corresponden a los siguientes valores:

$$5_d = \begin{matrix} A_2 & A_2 & A_1 & A_0 \\ 0 & 1 & 0 & 1 \end{matrix}$$

$$2_d = \begin{matrix} B_2 & B_2 & B_1 & B_0 \\ 0 & 0 & 1 & 0 \end{matrix}$$

$$C_{-1} = 1$$

Reemplazando dichos valores en las entradas de la implementación realizada, queda de la siguiente manera:



En complemento a 2, el último bit de *carry* se descarta.

El *bit de signo* $S_3 = 0$ indica que el resultado es positivo y los demás, que la magnitud es 3 (011).

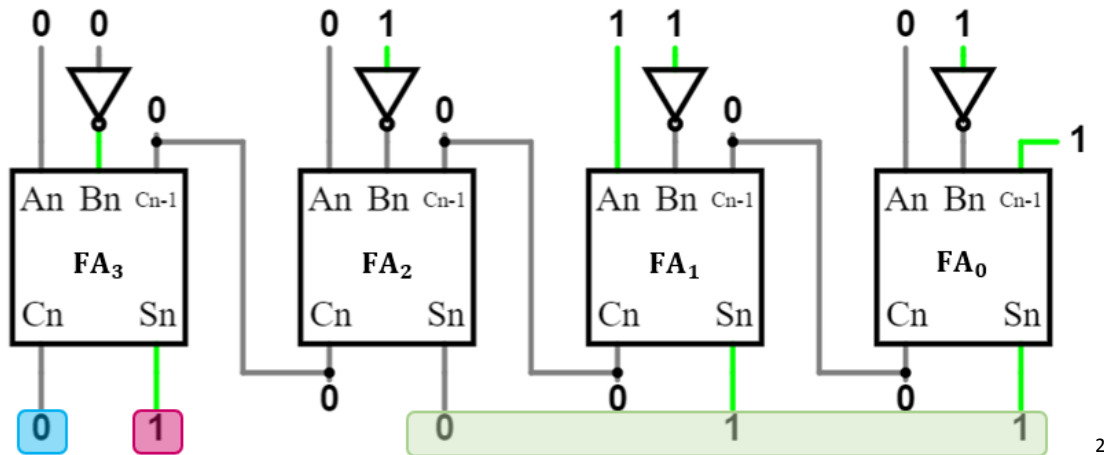
El resultado obtenido es el esperado: $5_d - 2_d = 3_d$.

Ejemplo 2: restador a partir de sumador en paralelo con carry en serie

Se quiere realizar una implementación que realice la siguiente operación $2_d - 7_d$ en binario.

Conociendo cómo se puede implementar esta operación, a través del uso de sumadores completos y alguna lógica adicional, tal como se vio en el ejemplo anterior, se llegaría al siguiente resultado:

$$\begin{array}{rcl}
 2_d = 0010_b & \begin{array}{c} A_2 \quad A_2 \quad A_1 \quad A_0 \\ 2_d = 0 \quad 0 \quad 1 \quad 0 \end{array} \\
 7_d = 0111_b & \begin{array}{c} B_2 \quad B_2 \quad B_1 \quad B_0 \\ 7_d = 0 \quad 1 \quad 1 \quad 1 \end{array} \\
 & C_{-1} = 1
 \end{array}$$



¿Es este el resultado esperado?

$S_3 = 1$ indica que el **signo** es negativo.

Para saber el valor de la magnitud antes de pasarlo a decimal, será necesario realizar el complemento a 2:

$$011 \rightarrow \text{complementando} \rightarrow 101$$

Pasando el resultado a decimal $101_b = 5_d$

Entonces, el resultado obtenido $S = S_3 S_2 S_1 S_0$ es -5_d .

Se verifica que este es el resultado esperado: $2_d - 7_d = -5_d$.

¿Se podría realizar una única implementación que pueda realizar una suma de dos números positivos y una suma de un número positivo y uno negativo (es decir, que pueda sumar o restar)?

Sí. A continuación, se verá la implementación de un sumador – restador.

Sumador – Restador (con *carry* en serie)

El sumador – restador con una única implementación podrá sumar o restar dos números solo cambiando los valores de las entradas.

Ejemplo 1: sumador – restador a partir de sumador en paralelo con carry en serie

Se quiere realizar una implementación que realice las siguientes operaciones:

$5_d + 7_d$ y $5_d - 7_d$ en binario.

Se tomará como A al número positivo en ambas operaciones, en este caso, 5, y como B al número que según la operación aritmética que se desea realizar es positivo o negativo, en este caso, 7.

Si se tiene en cuenta tanto la implementación que se usó para sumar dos números positivos como dos números de signo opuesto, para ambas es posible usar sumadores completos y el siguiente planteo:

$$\begin{array}{rcccc} & A_2 & A_2 & A_1 & A_0 \\ 5_d = & 0 & 1 & 0 & 1 \\ & B_2 & B_2 & B_1 & B_0 \\ 7_d = & 0 & 1 & 1 & 1 \end{array}$$

¿En qué difieren?

Por un lado, en que $C_{-1} = 0$, si se suman dos números positivos, mientras que en la resta $C_{-1} = 1$. Entonces, el valor de C_{-1} variará según se esté restando o sumando para esta implementación.

Por otro lado, será necesario pensar una lógica adicional que logre hacer que cuando se quiera sumar el valor que entre en B_n sea el dado, mientras que si se quiere restar el valor que entre en B_n del sumador sea el inverso.

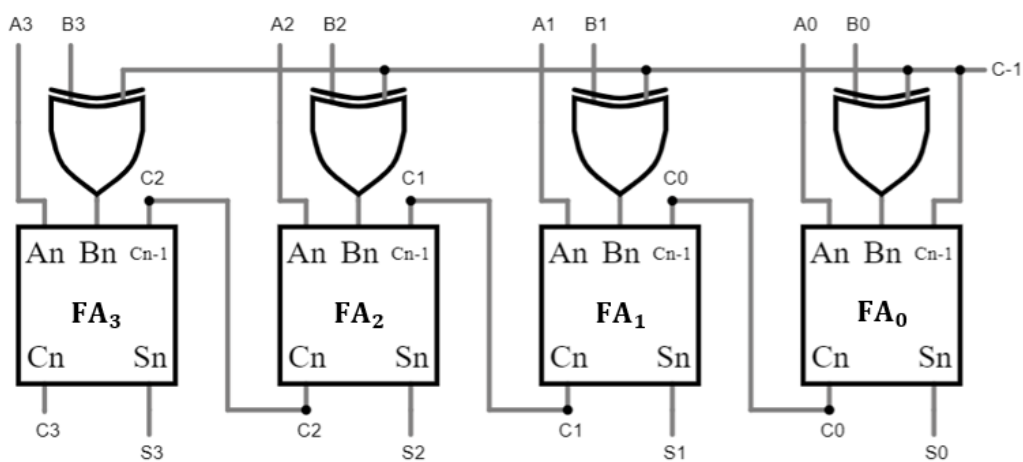
Anteriormente, se había resuelto el inverso con una compuerta *NOT*, pero para que funcione en ambos casos se debe pensar algo distinto.

Seguidamente, se plantea una tabla con todos los datos que corresponden según se sumen o resten dos números.

	C_{-1}	B	I
Suma	0	0	0
	0	1	1
Resta	1	0	1
	1	1	0

I : es el valor que se quiere obtener a la entrada del sumador, según se esté sumando o restando y según el valor de B_n .

Si se observa detenidamente la tabla, se puede ver que se corresponde con el comportamiento de la compuerta $X - OR$ (OR exclusiva). Por lo que se deberá agregar a la entrada de B_n de cada sumador una compuerta $X - OR$, cuyas entradas sean los valores de B_n correspondientes al número indicado y C_{-1} , cuyo valor será 1 o 0 según se este sumando o restando. Por lo cual, la implementación de un circuito sumador – restador será la siguiente:



2

Reemplazo para el caso de la suma ($5_d + 7_d$)

Se reemplazan los valores de las entradas por los siguientes:

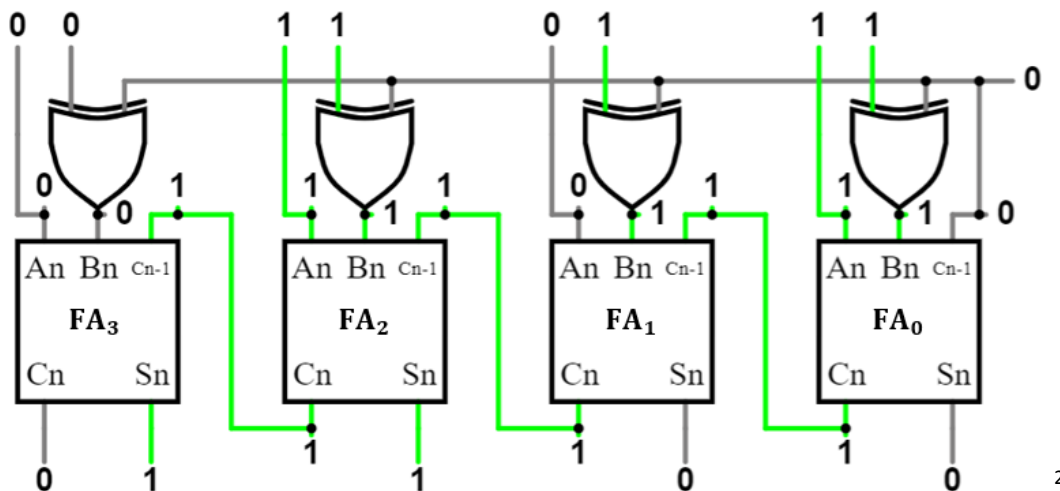
$$5_d = \begin{matrix} A_2 & A_2 & A_1 & A_0 \\ 0 & 1 & 0 & 1 \end{matrix}$$

$$7_d = \begin{matrix} B_2 & B_2 & B_1 & B_0 \\ 0 & 1 & 1 & 1 \end{matrix}$$

$$C_{-1} = 0$$

Las operaciones que hace cada uno de los sumadores según lo que recibe a la entrada

$$\begin{array}{r} 1 \quad 1 \quad 1 \quad 0 \\ 0 \quad 1 \quad 0 \quad 1 \\ + \quad 0 \quad 1 \quad 1 \quad 1 \\ \hline 01 \quad 11 \quad 10 \quad 10 \end{array}$$



Como se está en una suma, si se considera que la cuenta se hizo sin signo se puede considerar que el resultado es $S = S_3 S_2 S_1 S_0$ es decir $S = 1100_b = 12_d$

El resultado obtenido es el esperado: $5_d + 7_d = 12_d$.

Sin embargo, es interesante remarcar que, si se considera que es una suma con signo en complemento a 2, dado que el bit correspondiente al signo es 1, es decir es negativo, cuando se esperaría uno positivo, estaríamos ante un overflow (desbordamiento). Para solucionarlo se debería agregar un sumador más o, como antes, plantearlo como una suma sin signo, de esta manera nos mantenemos dentro del rango de representación.

Reemplazo para el caso de la resta ($5_d - 7_d$)

Se reemplazan los valores de las entradas por los siguientes:

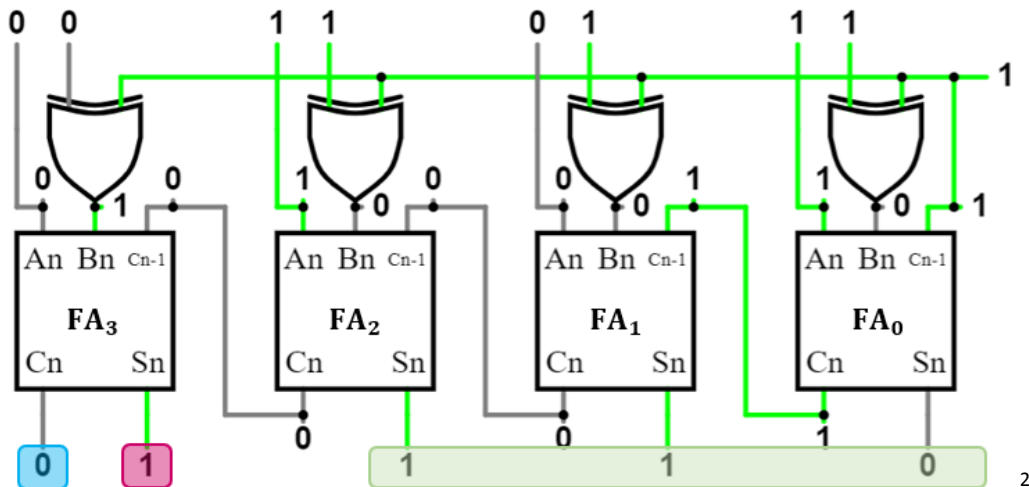
$$5_d = \begin{matrix} A_2 & A_2 & A_1 & A_0 \\ 0 & 1 & 0 & 1 \end{matrix}$$

$$7_d = \begin{matrix} B_2 & B_2 & B_1 & B_0 \\ 0 & 1 & 1 & 1 \end{matrix}$$

$$C_{-1} = 1$$

Las operaciones que hace cada uno de los sumadores según lo que recibe a la entrada:

$$+ \begin{array}{r} 0 \\ 0 \\ 1 \\ \hline 01 \end{array} + \begin{array}{r} 0 \\ 1 \\ 0 \\ \hline 01 \end{array} + \begin{array}{r} 1 \\ 0 \\ 0 \\ \hline 01 \end{array} + \begin{array}{r} 0 \\ 1 \\ 1 \\ \hline 10 \end{array}$$



En complemento a 2, el último bit de *carry* se descarta.

El *bit de signo* $S_3 = 1$ indica que el resultado es negativo.

Para obtener el valor de la magnitud es necesario complementar:

$$110 \rightarrow \text{complementando} \rightarrow 010$$

Pasando el resultado a decimal $010_b = 2_d$.

Entonces, el resultado obtenido $S = S_3 S_2 S_1 S_0 = 1110$ es -2_d .

Se verifica que este es el resultado esperado: $5_d - 7_d = -2_d$.

Sumadores

Hasta aquí se vio cómo se podrían usar los sumadores completos (*Full Adder*) para realizar sumadores que puedan sumar o restar números con más de un *bit*. Una manera de hacerlo es utilizando los sumadores en paralelo y conectando los *carry* en serie. Sin embargo, los *sumadores en paralelo con carry en serie* no son la única manera de realizar este tipo de implementación. Existen otras, como, por ejemplo, el *sumador en paralelo con carry anticipado*.

El sumador en paralelo con *carry* en serie no es tan veloz, pues debe esperar a que el primer sumador haga la cuenta completa, lo cual, como se ha visto, conlleva un retardo. Luego pasa el resultado del *carry* al siguiente y este puede, recién en ese momento, realizar la cuenta, que conlleva otro retardo. Es decir que un sumador de estas características irá acumulando esta propagación del tiempo que dependerá de la cantidad de *bits* (de sumadores) que contenga.

Por otro lado, el sumador en paralelo con *carry* anticipado, tal como lo dice su nombre, lleva una lógica más compleja, capaz de realizar el cálculo de manera anticipada (en paralelo), haciendo que no se acumule el retardo por *carry* de cada sumador que se use; es decir, independiza el retardo de la cantidad de *bits* en la cuenta, por lo que se puede decir que es más eficiente que el anterior.